Warm Up

What do we do when things go wrong in our code?

Objectives for Today

Data Types & Operations

Review existing types and expand on specialized operations.

Debugging Process

Master the general workflow for identifying and fixing errors.

Debug Activity

Hands-on practice applying the debug process in real scenarios.

Mini-Project Launch

Introduction to the unit's culminating mini-project task.

Python Data Types & Compatibility

In Python, data types determine how values behave and interact with each other. Some data types can be combined (or “operated on”) together, while others cannot. When combining data types, it’s important to understand their compatibility and how to handle mismatches.

1. Seamless Combinations

Same types interact easily:

- **Numbers:** Integers and floats work together for math.
- **Strings:** Combined via the + operator.

2. Type Conversion

Cross-type interaction:

Some combinations aren't allowed directly. Python provides functions to convert between types before operating.

3. Unsupported Pairs

Incompatible types:

Trying to combine incompatible types without conversion will trigger a `TypeError`.

Code Examples: Combining Types

Allowed: Int + Float

Integers and floats can be combined directly for mathematical operations.

```
# Output: 7.5
x = 5 # int
y = 2.5 # float

result = x + y
print(result)
```

Not Allowed: Int + String

Combining numbers and strings without explicit conversion results in an error.

```
# TypeError: unsupported...
x = 5 # int
y = "2" # str

result = x + y
```

String Concatenation

Allowed: Strings can be combined using the + operator.

```
greeting = "Hello"
name = "Alice"
message = greeting + " " + name
print(message) # Output: Hello Alice
```

Not Allowed: Strings + Numbers

Combining directly causes an error:

```
age = 25
message = "I am " + age
# TypeError: can only concatenate str...
```

Solution: Type Casting

Convert numbers to strings with str():

```
age = 25
message = "I am " + str(age)
print(message) # Output: I am 25
```

Notice that wrapping the 25 in str() makes it a string data type. Converting types is essential when inputs differ from program requirements.

Type Conversion

You can explicitly convert one type to another to enable combinations using built-in Python functions.

Conversion Functions

- **int()**: Converts a value to an integer.
- **float()**: Converts a value to a float.
- **str()**: Converts a value to a string.

Code Example

```
x = "10" # str
y = 5 # int

# Convert x to int before adding
result = int(x) + y

print(result) # Output: 15
```

Type casting ensures that data types are compatible before performing operations, preventing potential TypeErrors.

Unsupported Combinations

Some data types, like lists and strings, cannot be directly combined. Python requires explicit conversion to ensure compatibility.

The Problem: Direct Addition

```
my_list = [1, 2, 3]
my_string = "hello"
result = my_list + my_string
# TypeError: can only concatenate list (not "str") to
list
```

The Solution: list() conversion

```
my_list = [1, 2, 3]
my_string = "hello"
result = my_list + list(my_string)
print(result) # [1, 2, 3, 'h', 'e', 'l', 'l',
'o']
```

Note: Converting a string to a list breaks the string into individual character elements.

Key Takeaways

1. Be Mindful of Data Types

Python is strict about combining incompatible types. Always know what kind of data you are working with.

2. Use Type Conversion

When combining different types, convert them explicitly using functions like int(), float(), or str().

3. Error Messages Are Your Friend

If you see a TypeError, check your data types. Don't be afraid to search for errors online; most programmers have faced the same hurdles.

While AI can fix code, true mastery comes from understanding **why** something doesn't work. Be patient and do your own legwork.

Error Analysis in Coding

Debugging is an essential skill. Errors are a normal part of the learning process—even for experts.

Link: Errors and Debugging Guide

1. Identify

Try your code and look for errors. Python provides messages to help pinpoint issues.

2. Interpret

Understand different types of errors. Recognize them as opportunities to learn.

3. Fix

Apply strategies and tools to debug. Mastery comes from understanding why code fails.

In this section, we'll explore error types, interpretation, and fixing strategies using Python's built-in tools.

Types of Programming Errors

Let's go through the different types of errors you can expect to see in programming.

1. Syntax Errors

Grammar mistakes that break Python's rules.

Example:

```
print("Hello, world!
```

```
Msg: SyntaxError: EOL while scanning string literal
```

Fix:
Ensure quotes and parentheses are balanced.

2. Runtime Errors

Exceptions that occur while the program is running.

Example:

```
num = 5 / 0
```

```
Msg: ZeroDivisionError: division by zero
```

Fix:
Check input values to avoid illegal operations.

3. Logical Errors

The program runs but gives incorrect results.

Example:

```
total = 5 * 2 + 3
```

```
Output: 13 (Expected: 25)
```

Fix:
Use parentheses to clarify order of operations.

Debugging involves identifying, interpreting, and fixing these common hurdles to ensure program accuracy.

How to Fix Errors in Your Code

1. Read Carefully

Pay attention to the error type and line number where the error occurred.

2. Check for Typos

Python is case-sensitive. Ensure variable names are spelled exactly as defined.

3. Use Print Statements

Check variable values if the program gives wrong output.

```
# Debugging Example
print("Before: x =", x, "y =", y)
total = x - y # Meant to add x + y
print("Total:", total)
```

4. Test in small parts

5. Search resources online

6. Ask for help

Final Tip: Learn from Your Errors!

The Learning Process

Making mistakes is part of the learning process. The more errors you encounter and fix, the better you'll become at debugging your code.

Pro-Tip: Use print()

If your program runs but gives the wrong output, use print() to check variable values.

Remember: Every bug you fix is a step toward becoming a master programmer.

Activity - Debugging Detective

Activity Overview

Welcome to your first activity! In this assignment, you will demonstrate your understanding of Python basics by:

- Writing a simple Python program.
- Intentionally introducing a few errors into your code.
- Documenting the process of how to debug and fix those errors.

Goal: Show your coding skills and ability to think critically about solving problems.

Project Goals

By completing this project, you will:

1. Writing Structures

Write Python code that includes basic structures (variables, input/output, loops, and conditionals).

2. Error Identification

Identify and interpret error messages in Python to understand what went wrong.

3. Debug & Explain

Debug your own code and explain the process of resolving issues effectively.

Focus on practical application and critical thinking during the project.

Project Directions

1. Write a Python Program

- Input-driven (e.g., name/age).
- Use 'if', loops, and math/strings.
- Save as debug_project.py.

2. Introduce Intentional Errors

Add three distinct errors:

- Syntax (missing colons)
- Logical (wrong calculation)
- Runtime (division by zero)

3. Document the Process

- Take screenshots of error messages.
- For each error, record: Type, Cause, and Fix.
- Use a Jupyter Notebook with Markdown.

```
# Example Documentation Structure
## Error 1: SyntaxError
Cause: Missing ":" in for loop
Solution: Added colon at end of line
```

4. Submit: Corrected Program + Jupyter Notebook + Brief Reflection

Instruction - Pre-project Guidance

Mini Project Overview

As a close-out to this first unit, you've got a mini project where you will analyze some texts. In future units, you might have to do some research of techniques and gain your own understanding but to help you in this endeavor in this instruction you'll see some helpful and necessary techniques for the project.

Text Analysis with Python

Before you start your Text Analysis Project, you need to learn some important techniques for handling and analyzing text in Python. This guide will walk you through the key skills you'll use in your project, with explanations and examples.

This project provides extra guidance to bridge the gap between your ability and expectations, showing what's capable with Python.

1. Importing a Text File into Jupyter Notebook

To analyze a text file, you'll first need to import it into your Jupyter Notebook.

1. Upload to GitHub

Click on the "Upload" button in GitHub Codespaces and add your .txt file.

2. Read in Python

```
with open("filename.txt", "r") as file:
    text = file.read()
```

- Replace "filename.txt" with actual name.
- Stores file in 'text' variable.

3. Verify Output

```
print(text[:500])
```

Prints first 500 characters to verify the load was successful.

Follow these steps sequentially to prepare your text data for analysis.

2. & 3. Text Processing Techniques

Prepare your text for analysis by splitting it into words and ensuring case consistency.

2. Breaking into a List of Words

To analyze text, you often need to split it into individual words.

```
words = text.split()
print(words[:20]) # First 20
```

Splits at spaces and returns a list.

3. Converting to Lowercase

Python is case-sensitive ("Hello" vs "hello"). Convert for consistency.

```
text = text.lower()
```

"The" and "the" will now be treated as the same word.

These steps ensure your data is clean and ready for frequency analysis.

4. & 5. Advanced Text Analysis

Refine your text data by removing noise and calculating word occurrences.

4. Removing Punctuation

Punctuation marks don’t add meaning in most text analysis tasks.

```
import string
text = text.translate(str.maketrans("", "", string.punctuation))
```

- Removes commas, periods, etc.
- Uses string.punctuation constant.

5. Counting Word Frequency

Use a dictionary to count how often each word appears in the text.

```
# Create empty dictionary
word_counts = {}
for word in words:
        word_counts[word] = word_counts.get(word, 0) + 1
```

This loop efficiently maps each word to its frequency count.

These final steps complete the core data preparation for your text analysis project.

6. Finding the Most Common Words

To see which words appear most frequently, sort the dictionary by value.

Sorting by Frequency

```
sorted_words = sorted(word_counts.items(), key=lambda x: x[1], reverse=True)
print(sorted_words[:10]) # Print the 10 most common words
```

This sorts words by frequency from highest to lowest.

Summary of Key Techniques: These foundational text analysis techniques will help you complete your project successfully!

Technique	Code Example
Importing text	`with open("file.txt", "r") as file: text = file.read()`
Splitting into words	`words = text.split()`
Lowercasing text	`text = text.lower()`
Removing punctuation	`text = text.translate(str.maketrans("", "", string.punctuation))`
Counting words	`word_counts[word] = word_counts.get(word, 0) + 1`
Finding most common words	`sorted(word_counts.items(), key=lambda x: x[1], reverse=True)`

Project: Text Analysis with Python

Project Overview

In this project, you will use Python loops and string manipulation to analyze a famous author's text. Apply your Unit 1 skills to count words, identify patterns, and summarize real-world text data.

Project Goals

Process Text Data

Use loops to efficiently process and analyze large text datasets.

Master Logic

Practice complex string manipulation and conditional logic.

Real-World Problem Solving

Apply your Python skills to solve tangible, data-driven problems.

Reflect on Automation

Understand the power of automating large-scale data analysis.

Project Instructions: Text Analysis

Follow these steps to complete your Python text analysis project.

1. Choose a Text

- Select an online source (Project Gutenberg or excerpt).
- Save as .txt in GitHub Codespace.

2. Write Python Program

Core functionality requirements:

- Read file & print total word count.
- Count specific word occurrences.
- Identify most frequent words.
- *(Optional) Count sentences.*

3. Loops & Logic

- Use for/while loops to process text.
- Use conditionals for pattern matching.

4. Test Program

- Verify results on chosen text.
- Add explanatory comments.

5. Submit Work

- Upload Python file to GitHub.
- Write reflection (1-2 paragraphs).

Ensure all requirements are met before final submission to your repository.